

NLP Pipeline

The whole is more than the sum of its parts. It is more correct to say that the whole is something else than the sum of its parts, because summing up is a meaningless procedure, whereas the whole-part relationship is meaningful.
Kurt Koffka

In the previous chapter, we saw examples of some common NLP applications that we might encounter in everyday life. If we were asked to build such an application, think about how we would approach doing so at our organization. We would normally walk through the requirements and break the problem down into several sub-problems, then try to develop a step-by-step procedure to solve them. Since language processing is involved, we would also list all the forms of text processing needed at each step. This step-by-step processing of text is known as a *pipeline*. It is the series of steps involved in building any NLP model. These steps are common in every NLP project, so it makes sense to study them in this chapter. Understanding some common procedures in any NLP pipeline will enable us to get started on any NLP problem encountered in the workplace. Laying out and developing a text-processing pipeline is seen as a starting point for any NLP application development process. In this chapter, we will learn about the various steps involved and how they play important roles in solving the NLP problem and we'll see a few guidelines about when and how to use which step. In later chapters, we'll discuss specific pipelines for various NLP tasks (e.g., Chapters 4–7).

[Figure 2-1](#) shows the main components of a generic pipeline for modern-day, data-driven NLP system development. The key stages in the pipeline are as follows:

1. Data acquisition
2. Text cleaning
3. Pre-processing
4. Feature engineering
5. Modeling
6. Evaluation
7. Deployment
8. Monitoring and model updating

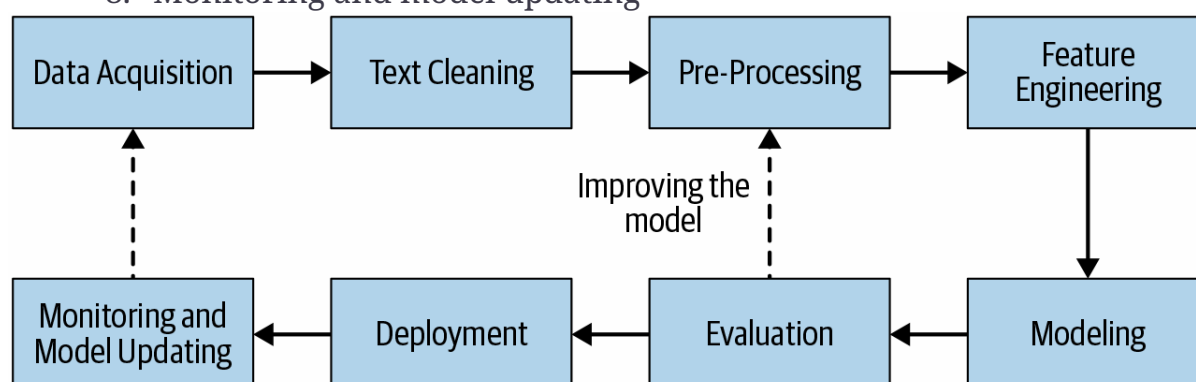


Figure 2-1. Generic NLP pipeline

The first step in the process of developing any NLP system is to collect data relevant to the given task. Even if we're building a rule-based system, we still need some data to design and test our rules. The data we get is seldom clean, and this is where text cleaning comes into play. After cleaning, text data often has a lot of variations and needs to be converted into a canonical form. This is done in the pre-processing step. This is followed by feature engineering, where we carve out indicators that are most suitable for the task at hand. These indicators are converted into a format that is understandable by modeling algorithms. Then comes the modeling and evaluation phase, where we build one or more models and compare and contrast them using a relevant evaluation metric(s). Once the best model among the ones evaluated is chosen, we move toward deploying this model in production. Finally, we regularly monitor the performance of the model and, if need be, update it to keep up its performance.

Note that, in the real world, the process may not always be linear as it's shown in the pipeline in [Figure 2-1](#); it often involves going back and forth between individual steps (e.g., between feature extraction and modeling, modeling and evaluation, and so on). Also, there are loops in between, most commonly going from evaluation to pre-processing, feature engineering, modeling, and back to evaluation. There is also an overall loop that goes from monitoring to data acquisition, but this loop happens at the project level.

Note that exact step-by-step procedures may depend on the specific task at hand. For example, a text-classification system may require a different feature extraction step compared to a text-summarization system. We will focus on application-specific pipeline stages in subsequent chapters in the book. Also, depending on the phase of the project, different steps can take different amounts of time. In the initial phases, most of the time is used in modeling and evaluation, whereas once the system matures, feature engineering can take far more time.

Step-by-step procedures of pre-processing in each case, as summarized in [Figure 2-11](#).

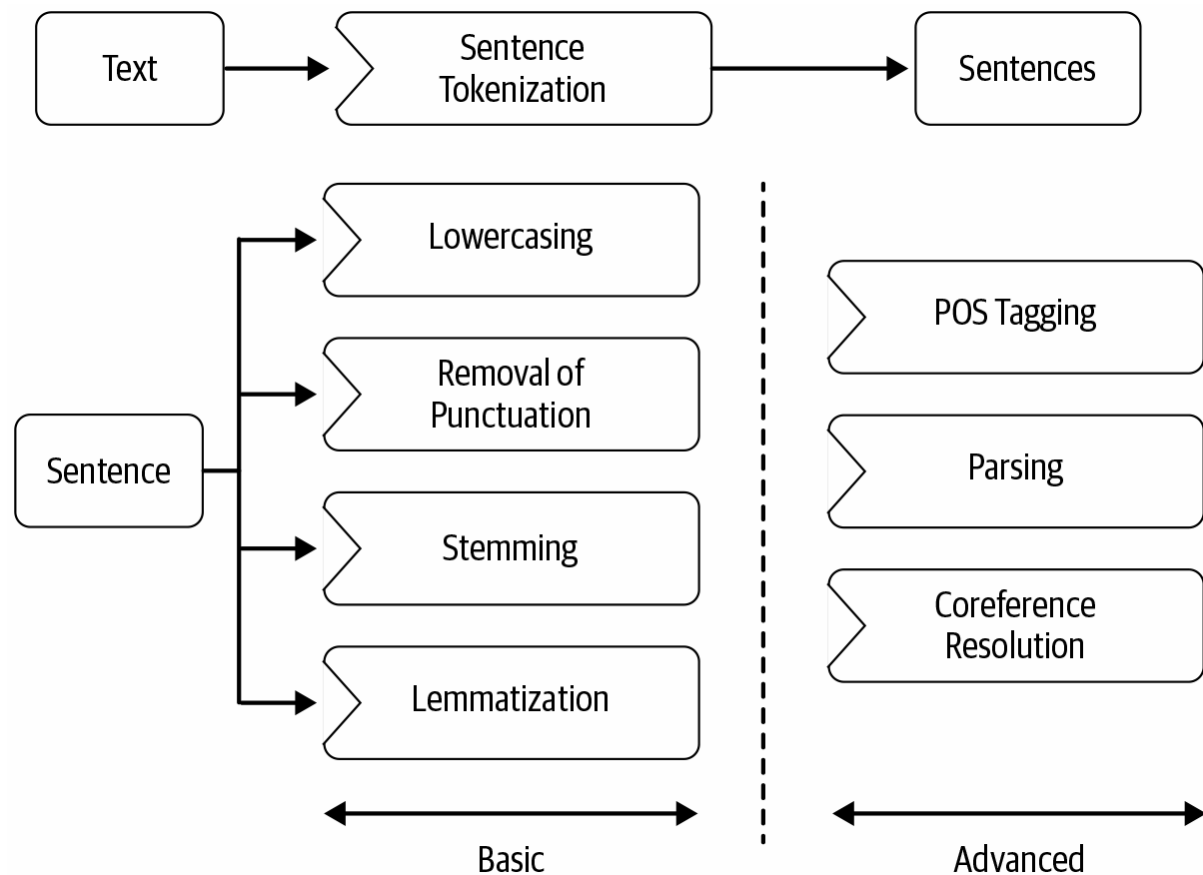


Figure 2-11. Advanced pre-processing steps on a blob of text

For example, POS tagging cannot be preceded by stop word removal, lowercasing, etc., as such processing affects POS tagger output by changing the grammatical structure of the sentence. How a particular pre-processing step is helping a given NLP problem is another question that is specific to the application, and it can only be answered with a lot of experimentation. We'll discuss more specific pre-processing required for different NLP applications in upcoming chapters. For now, let's move on to the next step: feature engineering.